COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/31/26	**DATE SUBMITTED:**	7/31/26
NAME:	Reyes, Cyrill Joferson B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

B.2. Stacks using Linked Lists

```
#include <iostream>

class Node {
public:
    int data;
    Node* next;
};

Node *head = NULL, *tail = NULL;

void push(int newData) {
    Node *newNode = new Node;
```

T.I.P COMPUTER ENGINEERING QUEZON CITY

```
    newNode->data = newData;
    newNode->next = head;

    if (head == NULL) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head = newNode;
    }
}

int pop() {
    int tempVal;
    Node *temp;

    if (head == NULL) {
        head = tail = NULL;
        std::cout << "Stack Underflow." << std::endl;
        return -1;
    } else {
        temp = head;
        tempVal = temp->data;
        head = head->next;
        delete temp;
        return tempVal;
    }
}

void Top() {
    if (head == NULL) {
        std::cout << "Stack is Empty." << std::endl;
        return;
    } else {
        std::cout << "Top of Stack: " << head->data << std::endl;
    }
}

int main() {
    push(1);
    std::cout << "After the first PUSH top of stack is: ";
    Top();

    push(5);
    std::cout << "After the second PUSH top of stack is: ";
    Top();

    pop();
    std::cout << "After the first POP operation, top of stack is: ";
    Top();
```

Tasks:

- Modify the code given above to include a function that will display all elements in the stack.
- Provide a description of each operation provided.
- Include your output in the output section of your activity template.

4. SUPPLEMENTARY ACTIVITY

ILO C: Solve problems using an implementation of stack:

The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:

a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL

Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.

Steps:
1. Create a Stack.
2. While(end of input is not reached) {
   a) If the character read is not a symbol to be balanced, ignore it.
   b) If the character is an opening symbol, push it onto the stack.
   c) If it is a closing symbol:
      i) Report an error if the stack is empty.
      ii) Otherwise, pop the stack.
   d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

Self-Checking:
For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
((A+B)+[C-D])			
((A+B]+[C-D]}			

COMPUTER ENGINEERING
QUEZON CITY

Tools Analysis:

- How do the different internal representations affect the implementation and usage of the stack?

5. Conclusions/Learnings:

Provide the following:

- Summary of lessons learned
- Analysis of the procedure
- Analysis of the supplementary activity
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
49  // Top: Displays the value at the top of the stack
50  template <typename T>
51  void Top() {
52      if (isEmpty<T>()) {
53          std::cout << "Stack underflow\n";
54          return;
55      }
56      std::cout << "Top of Stack: " << head<T>->data << std::endl;
57  }
58
59  // displayALL: Traverses and displays all elements in the stack
60  template <typename T>
61  void displayALL() {
62      if (head<T> == nullptr) {
63          std::cout << "stack underflow\n";
64          return;
65      }
66      std::cout << "Displaying the stack:\n";
67      Node<T>* temp = head<T>;
68      while (temp != nullptr) {
69          std::cout << temp->data << "\n";
70          temp = temp->next;
71      }
72      std::cout << "\n";
73  }
74
75  #endif
```

HOAB.2 > C stackList.h > Node<T>

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

#include <iostream>

// Class Node that contains data and pointer to the next node
template <typename T>
class Node {
public:
    T data;
    Node* next;
};

// Pointer for the top of the stack
template <typename T>
Node<T>* head = nullptr;

// STACK OPERATIONS

// push: Adds an element to the top of the stack
template <typename T>
void push(T newData) {
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

// pop: Removes and returns the top element of the stack
template <typename T>
T pop() {
    if (head<T> == nullptr) {
        std::cout << "Stack underflow!\n";
        return T();
    }
    Node<T>* temp = head<T>;
    T tempVal = temp->data;
    head<T> = head<T>->next;
    delete temp;
    return tempVal;
}

// isEmpty: Checks if the stack is empty
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

```

```cpp
// Top: Displays the value at the top of the stack
template <typename T>
void Top() {
    if (isEmpty<T>()) {
        std::cout << "Stack underflow\n";
        return;
    }
    std::cout << "Top of Stack: " << head<T>->data << std::endl;
}

// displayALL: Traverses and displays all elements in the stack
template <typename T>
void displayALL() {
    if (head<T> == nullptr) {
        std::cout << "stack underflow\n";
        return;
    }
    std::cout << "Displaying the stack:\n";
    Node<T>* temp = head<T>;
    while (temp != nullptr) {
        std::cout << temp->data << "\n";
        temp = temp->next;
    }
    std::cout << "\n";
}

#endif
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\OneDrive\Documents\CPE12S4 - Reyes>  & 'c:\Use
tdout=Microsoft-MIEngine-Out-5nyrw0sf.esu' '--stderr=Microsoft-M
--- Testing Stack Operations ---

Top of Stack: 6
Displaying the stack:
6
14
5

Popped value: 6

Displaying the stack:
14
5

PS C:\Users\TIPQC\OneDrive\Documents\CPE12S4 - Reyes>
```

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, code debugging, response formatting, concept explanation, and drafting structured summaries for the activity template (Tools Analysis, Conclusions, and Learnings).**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used as an interactive academic assistant to help modify C++ stack code, debug algorithm logic for symbol balancing, organize template output tables, and format analytical responses. The generated code and summaries were reviewed, tested, and validated to ensure complete understanding and personal authorship of the final submission without replacing original thinking.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.